Data Generating Function

Mark Fulginiti

2026-04-01

A Modular Data-Generating Function for Simulation Studies

Description:

Generate a flexible simulated dataset with optional complications commonly used in applied statistical work, including correlated predictors, nonlinear signal, interactions, observational treatment assignment, overlap problems, heteroskedasticity, missingness, and measurement error. The function is designed to support a wide range of technical notes and mini papers using a common data-generating framework.

Usage:

generate_sim_data(

```
n = 1000,
seed = NULL,
outcome_type = c("continuous", "binary", "count", "multinomial"),
correlation = c("moderate", "none", "strong"),
nonlinearity = c("none", "moderate", "strong"),
interactions = c("none", "moderate", "strong"),
confounding = c("none", "moderate", "strong"),
overlap = c("good", "moderate", "poor"),
heteroskedasticity = c("none", "moderate", "strong"),
missingness = c("none", "mar_moderate", "mar_strong"),
measurement_error = c("none", "moderate", "strong"),
return_potential_outcomes = TRUE
)
```

Arguments:

n: Integer. Number of observations to generate.

seed: Optional integer seed for reproducibility. If ‘NULL’, no seed is set.

outcome_type: Character string specifying the outcome family. Available options are:

```
- "continuous" for a Gaussian-type continuous outcome
- "binary" for a Bernoulli outcome
- "count" for a Poisson count outcome
- "multinomial" for a three-category nominal outcome
```

correlation: Character string controlling correlation among the continuous covariates. Available options are:

```
- "none" for no correlation
- "moderate" for moderate pairwise correlation
- "strong" for strong pairwise correlation
```

nonlinearity: Character string controlling nonlinear structure in the outcome model. Available options are:

```
- "none" for a purely linear signal
- "moderate" for modest nonlinear terms
- "strong" for stronger nonlinear structure
```

interactions: Character string controlling interaction effects in the outcome model. Available options are:

```
- "none" for no interactions
- "moderate" for modest interaction effects
- "strong" for stronger interaction effects
```

confounding: Character string controlling the degree to which treatment assignment depends on pre-treatment covariates. Available options are:

```
- "none" for randomized treatment assignment
- "moderate" for moderate confounding
- "strong" for stronger confounding
```

overlap: Character string controlling the quality of overlap in the treatment assignment mechanism. Available options are:

```
- "good" for propensity scores concentrated away from 0 and 1
- "moderate" for less stable overlap
- "poor" for more extreme propensity scores and weaker common support
```

heteroskedasticity: Character string controlling whether the continuous outcome has non-constant error variance. Available options are:

```
- "none" for homoskedastic errors
- "moderate" for moderate heteroskedasticity
- "strong" for stronger heteroskedasticity
```

Ignored for binary, count, and multinomial outcomes.

missingness: Character string controlling missing-data generation. Available options are:

```
- "none" for no missingness
- "mar_moderate" for moderate missingness under a MAR-like mechanism
- "mar_strong" for stronger missingness under a MAR-like mechanism

In the current implementation, missingness may affect the
observed outcome and one observed covariate.
```

measurement_error: Character string controlling additive measurement error in selected continuous covariates. Available options are:

```
- "none" for no measurement error
- "moderate" for moderate additive noise
- "strong" for stronger additive noise
```

return_potential_outcomes: Logical. If ‘TRUE’, return an additional full-data object containing the true covariates, treatment assignment, potential outcomes, realized outcome, and individual treatment effect.

Details:

```
The function builds the simulated data in several layers.

1. Covariates:
   A base covariate set is generated, including three continuous
   covariates, one binary covariate, one ordinal-style covariate,
   and one auxiliary covariate (x6) used to help control overlap
   independently of confounding. Correlation among the continuous
```

covariates is controlled by ‘correlation’.

2. Measurement error:
   If requested, additive error is applied to selected continuous covariates in the observed dataset. The true covariates remain available in the returned full-data object when ‘return_potential_outcomes = TRUE’.

3. Outcome signal:
   A baseline linear signal is constructed, then optionally modified by nonlinear terms and interaction terms according to ‘nonlinearity’ and ‘interactions’.

4. Treatment assignment:
   Treatment is assigned through a logistic model composed of two separable parts: a confounding component based on outcome-relevant covariates and an overlap component based on a separate non-prognostic covariate. The confounding argument controls the strength of selection on prognostic covariates, while the overlap argument controls how extreme the treatment probabilities become and therefore how weak common support is.

5. Potential outcomes:
   Potential outcomes ‘Y0’ and ‘Y1’ are generated using the latent untreated mean and a heterogeneous treatment effect. The realized observed outcome is then defined by the switching equation:
   Y = D Y(1) + (1 - D) Y(0).

6. Missingness:
   If requested, missingness is introduced through a MAR-like mechanism depending on observed covariates and treatment.

7. True estimands:
   The function computes the true sample ATE, ATT, and ATC from the simulated potential outcomes.

The design is intended as a reusable backbone for simulation-based applied work. Individual notes can hold most settings fixed and vary one or two arguments to isolate a specific methodological issue.

Mathematical Specification

1. Covariates

Let

$$(X_1, X_2, X_3)^\top \sim N_3(0, \Sigma_\rho),$$

where

$$\Sigma_\rho = \begin{pmatrix} 1 & \rho & \rho \\ \rho & 1 & \rho \\ \rho & \rho & 1 \end{pmatrix},$$

with $\rho \in \{0, 0.35, 0.7\}$ depending on the correlation setting.

To generate these correlated covariates in practice, let

$$Z_i \sim N_3(0, I_3), \qquad i = 1, \dots, n,$$

where the rows of Z are independent standard normal vectors. Let U be the Cholesky factor of Σ_ρ, so that

$$U^\top U = \Sigma_\rho.$$

Then define

$$X_i = Z_i U, \qquad i = 1, \dots, n.$$

Equivalently, in matrix form,

$$X = ZU.$$

This transformation preserves mean zero and induces the desired covariance structure, since

$$\mathrm{Cov}(X_i) = \mathrm{Cov}(Z_i U) = U^\top \mathrm{Cov}(Z_i) U = U^\top I_3 U = \Sigma_\rho.$$

Also define

$$X_4 \sim \mathrm{Bernoulli}(0.5),$$

$$X_5 \in \{0, 1, 2\}, \qquad P(X_5 = 0) = 0.3, \quad P(X_5 = 1) = 0.45, \quad P(X_5 = 2) = 0.25,$$

and

$$X_6 \sim N(0, 1).$$

2. Measurement Error

If measurement error is enabled, the observed covariates differ from the true covariates according to

$$X_1^{\text{obs}} = X_1 + \varepsilon_1, \qquad X_2^{\text{obs}} = X_2 + \varepsilon_2,$$

where

$$\varepsilon_j \sim N(0, \sigma_{me}^2),$$

and $\sigma_{me} \in \{0, 0.4, 0.8\}$ depends on the measurement-error setting.

3. Baseline Signal

The baseline linear signal is

$$f_{\text{base}}(X) = 0.8X_1 - 0.6X_2 + 0.5X_3 + 0.7X_4 - 0.3X_5.$$

4. Nonlinear Component

The nonlinear component is defined as follows.

For no nonlinearity,

$$f_{\mathrm{nl}}(X) = 0.$$

For moderate nonlinearity,

$$f_{\mathrm{nl}}(X) = 0.5X_1^2 - 0.4\sin(X_2).$$

For strong nonlinearity,

$$f_{\mathrm{nl}}(X) = 0.9X_1^2 - 0.7\sin(X_2) + 0.5\max(X_3, 0).$$

5. Interaction Component

The interaction component is defined as follows.

For no interactions,

$$f_{\mathrm{int}}(X) = 0.$$

For moderate interactions,

$$f_{\mathrm{int}}(X) = 0.6X_1X_4 - 0.4X_2X_3.$$

For strong interactions,

$$f_{\mathrm{int}}(X) = 1.0X_1X_4 - 0.7X_2X_3 + 0.5X_1X_5.$$

6. Untreated and Treated Latent Means

The untreated latent mean is

$$\mu_0(X) = f_{\mathrm{base}}(X) + f_{\mathrm{nl}}(X) + f_{\mathrm{int}}(X).$$

The heterogeneous treatment effect is

$$\tau(X) = 1.0 + 0.5X_1 - 0.4X_4 + 0.3\,\mathbf{1}(X_5 = 2).$$

The treated latent mean is

$$\mu_1(X) = \mu_0(X) + \tau(X).$$

7. Treatment Assignment

Treatment assignment is generated from a logistic model with separate confounding and overlap components. The confounding component is

$$\eta_{\text{conf}}(X) = s_{\text{conf}}\,(0.8X_1 - 0.6X_2 + 0.5X_4 - 0.3X_5),$$

where $s_{\text{conf}} \in \{0, 0.8, 1.5\}$ depends on the confounding setting.

The overlap component is

$$\eta_{\text{overlap}}(X) = s_{\text{overlap}}\,X_6,$$

where $s_{\text{overlap}} \in \{0, 0.8, 1.8\}$ depends on the overlap setting.

The full treatment-assignment linear predictor is

$$\eta_D(X) = \eta_{\text{conf}}(X) + \eta_{\text{overlap}}(X).$$

The propensity score is then

$$P(D = 1 \mid X) = \text{logit}^{-1}(\eta_D(X)),$$

and the observed treatment indicator is generated as

$$D \mid X \sim \text{Bernoulli}(P(D = 1 \mid X)).$$

8. Potential Outcomes by Outcome Family

Continuous Outcome For continuous outcomes,

$$Y(0) = \mu_0(X) + \epsilon_0, \qquad Y(1) = \mu_1(X) + \epsilon_1,$$

where

$$\epsilon_0, \epsilon_1 \sim N(0, \sigma^2(X)).$$

For homoskedastic errors,

$$\sigma(X) = 1.$$

For moderate heteroskedasticity,

$$\sigma(X) = \exp(0.35X_1).$$

For strong heteroskedasticity,

$$\sigma(X) = \exp(0.7X_1).$$

Binary Outcome For binary outcomes,

$$Y(0) \sim \text{Bernoulli}(\text{logit}^{-1}(\mu_0(X))),$$

$$Y(1) \sim \text{Bernoulli}(\text{logit}^{-1}(\mu_1(X))).$$

Count Outcome For count outcomes,

$$Y(0) \sim \text{Poisson}(\lambda_0(X)), \qquad Y(1) \sim \text{Poisson}(\lambda_1(X)),$$

where

$$\lambda_0(X) = \exp(\mu_0(X)/3),$$

$$\lambda_1(X) = \exp(\mu_1(X)/3).$$

The division by 3 in $\exp(\mu/3)$ is a scale-stabilizing choice that keeps Poisson means in a practically useful range across simulation settings with stronger signal components.

Multinomial Outcome For multinomial outcomes, define the untreated logits as

$$\eta_1^{(0)} = 0, \qquad \eta_2^{(0)} = 0.6\mu_0(X), \qquad \eta_3^{(0)} = -0.4\mu_0(X) + 0.5X_2 - 0.3X_4,$$

and the treated logits as

$$\eta_1^{(1)} = 0, \qquad \eta_2^{(1)} = 0.6\mu_1(X), \qquad \eta_3^{(1)} = -0.4\mu_1(X) + 0.5X_2 - 0.3X_4.$$

Then for $a \in \{0, 1\}$,

$$P(Y(a) = k \mid X) = \frac{\exp(\eta_k^{(a)})}{\sum_{j=1}^{3} \exp(\eta_j^{(a)})}, \qquad k = 1, 2, 3.$$

9. Observed Outcome

The observed outcome is defined by the switching equation

$$Y = D\,Y(1) + (1 - D)\,Y(0).$$

10. Missingness Mechanism

If missingness is enabled, define

$$P(R_Y = 1 \mid X, D) = \text{logit}^{-1}\big(-2 + \gamma(0.8X_1 + 0.6D)\big),$$

and

$$P(R_{X_2} = 1 \mid X, D) = \text{logit}^{-1}\big(-2.2 + \gamma(0.7X_4 + 0.5X_5)\big),$$

where $R = 1$ indicates that the value is missing, and $\gamma \in \{0.6, 1.1\}$ depends on the missingness setting. Observed values are then set to missing according to these indicators.

11. Truth Quantities

For non-multinomial outcomes, the true sample causal estimands are

$$ATE = \frac{1}{n}\sum_{i=1}^{n}\big(Y_i(1) - Y_i(0)\big),$$

$$ATT = \frac{1}{\sum_i D_i}\sum_{i:D_i=1}\big(Y_i(1) - Y_i(0)\big),$$

and

$$ATC = \frac{1}{\sum_i (1 - D_i)}\sum_{i:D_i=0}\big(Y_i(1) - Y_i(0)\big).$$

Value:

```
A list with the following components:

data:
     A data frame containing the observed dataset. This includes the
     observed covariates, treatment indicator 'D', true propensity
     score 'ps_true', and observed outcome 'Y'. Missingness and
     measurement error are reflected here if requested.

metadata:
     A named list recording the simulation settings used in the call.

truth:
     A named list containing the true sample causal estimands:

       - 'ate' = average treatment effect
       - 'att' = average treatment effect on the treated
       - 'atc' = average treatment effect on the controls

full_data:
     Returned only if 'return_potential_outcomes = TRUE'. A data
     frame containing the true underlying covariates, treatment
     assignment, propensity score, potential outcomes 'Y0' and 'Y1',
     realized outcome 'Y', and individual treatment effect ite: unit-level
     realized causal contrast Y1 - Y0 for non-multinomial outcomes.
```

Assumptions and Conventions:

```
The function is primarily designed for binary treatment settings.

The continuous outcome case uses Gaussian noise, optionally with
heteroskedastic variance.

The binary outcome case uses a Bernoulli model with probabilities
```

generated by a logistic transformation.

The count outcome case uses a Poisson model based on exponentiated latent means. The division by 3 in exp(mu/3) is a scale-stabilizing choice that keeps Poisson means in a practically useful range across simulation settings with stronger signal components.

For multinomial outcomes, scalar truth summaries such as ATE, ATT, ATC, and unit-level ite = Y(1) - Y(0) are not returned because the current implementation does not define a single scalar causal contrast for that outcome family.

Missingness is currently generated through simple MAR-like mechanisms based on covariates and treatment, rather than through a fully general missing-data process.

Confounding and overlap are modeled separately in the treatment-assignment mechanism. Confounding is induced by letting treatment depend on outcome-relevant covariates, while overlap is degraded by letting treatment also depend on a separate covariate excluded from the outcome model. As a result, poor overlap can occur even when treatment assignment is otherwise unconfounded, and confounding can be present even when overlap remains relatively good.

Note:

This function is intentionally modular rather than exhaustive. It is meant to be extended as new papers require additional features such as survival outcomes, clustered data, multivalued treatment, mediation structures, latent classes, or explicit selection models.

Author(s):

Mark Fulginiti

Examples:

```
## Basic continuous-outcome example
sim <- generate_sim_data(
  n = 500,
  seed = 123,
  outcome_type = "continuous"
)

head(sim$data)
sim$truth

## Observational causal example with confounding and moderate overlap
sim2 <- generate_sim_data(
  n = 1000,
  seed = 123,
  outcome_type = "continuous",
  confounding = "moderate",
  overlap = "moderate",
```

```
  return_potential_outcomes = TRUE
)

head(sim2$data)
head(sim2$full_data)
sim2$truth

## Binary outcome with nonlinear signal and missingness
sim3 <- generate_sim_data(
  n = 750,
  seed = 456,
  outcome_type = "binary",
  nonlinearity = "moderate",
  interactions = "moderate",
  missingness = "mar_moderate"
)

summary(sim3$data)

## Count outcome with stronger complications
sim4 <- generate_sim_data(
  n = 1000,
  seed = 789,
  outcome_type = "count",
  correlation = "strong",
  confounding = "strong",
  overlap = "poor",
  measurement_error = "moderate"
)

summary(sim4$data)
```

See Also:

'set.seed', 'glm', 'lm', 'rpois', 'rbinom', 'rnorm'

```r
generate_sim_data <- function(
  n = 1000,
  seed = NULL,
  outcome_type = c("continuous", "binary", "count", "multinomial"),
  correlation = c("moderate", "none", "strong"),
  nonlinearity = c("none", "moderate", "strong"),
  interactions = c("none", "moderate", "strong"),
  confounding = c("none", "moderate", "strong"),
  overlap = c("good", "moderate", "poor"),
  heteroskedasticity = c("none", "moderate", "strong"),
  missingness = c("none", "mar_moderate", "mar_strong"),
  measurement_error = c("none", "moderate", "strong"),
  return_potential_outcomes = TRUE
) {
  # Resolve each user-supplied option to one allowed value.
  # This guards against misspecified arguments and standardizes downstream logic.
  outcome_type <- match.arg(outcome_type)
  correlation <- match.arg(correlation)
```

```r
nonlinearity <- match.arg(nonlinearity)
interactions <- match.arg(interactions)
confounding <- match.arg(confounding)
overlap <- match.arg(overlap)
heteroskedasticity <- match.arg(heteroskedasticity)
missingness <- match.arg(missingness)
measurement_error <- match.arg(measurement_error)

# Set a random seed if requested so simulation results are reproducible.
if (!is.null(seed)) set.seed(seed)

# ---------------------------------------------------------------------------
# 0. Helper functions
# ---------------------------------------------------------------------------

# Logistic inverse link.
# Used for treatment probabilities, binary outcomes, and missingness models.
inv_logit <- function(z) 1 / (1 + exp(-z))

# Row-wise softmax transformation.
# Converts a matrix of category-specific linear predictors into multinomial
# probabilities that sum to 1 within each row.
# Subtracting the row maximum improves numerical stability.
softmax_vec <- function(eta_mat) {
  eta_shift <- eta_mat - apply(eta_mat, 1, max)
  exp_eta <- exp(eta_shift)
  exp_eta / rowSums(exp_eta)
}

# ---------------------------------------------------------------------------
# 1. Generate base covariates
# ---------------------------------------------------------------------------

# Map the requested correlation level to the pairwise correlation rho
# used in the 3 x 3 covariance matrix for the continuous covariates.
rho <- switch(
  correlation,
  "none" = 0.0,
  "moderate" = 0.35,
  "strong" = 0.7
)

# Covariance matrix for (x1, x2, x3).
Sigma <- matrix(
  c(1, rho, rho,
    rho, 1, rho,
    rho, rho, 1),
  nrow = 3, byrow = TRUE
)

# Draw independent standard normals and then induce the target covariance
# structure using a Cholesky transformation.
Z <- matrix(rnorm(n * 3), nrow = n, ncol = 3)
```

```r
X_cont <- Z %*% chol(Sigma)

# Store each continuous covariate as a numeric vector.
x1 <- as.numeric(X_cont[, 1])
x2 <- as.numeric(X_cont[, 2])
x3 <- as.numeric(X_cont[, 3])


# Additional covariates:
# x4: binary covariate
# x5: ordinal-style categorical covariate with three levels
# x6: auxiliary covariate that is noise for the outcome model but enters
# the treatment-assignment mechanism to vary overlap independently of confounding
x4 <- rbinom(n, size = 1, prob = 0.5)
x5 <- sample(0:2, size = n, replace = TRUE,
             prob = c(0.3, 0.45, 0.25))
x6 <- rnorm(n)

# True covariate data before any measurement error is introduced.
X_true <- data.frame(
  id = seq_len(n),
  x1 = x1, x2 = x2, x3 = x3, x4 = x4, x5 = x5, x6 = x6
)

# -----------------------------------------------------------------------------
# 2. Add optional measurement error to observed covariates
# -----------------------------------------------------------------------------

# Start the observed data as a copy of the truth, then perturb selected
# covariates if measurement error is requested.
X_obs <- X_true
if (measurement_error != "none") {
  # Standard deviation of additive measurement error.
  me_sd <- switch(
    measurement_error,
    "moderate" = 0.4,
    "strong" = 0.8
  )

  # Add classical measurement error to x1 and x2 in the observed data only.
  # The true values remain available in X_true and, optionally, full_data.
  X_obs$x1 <- X_true$x1 + rnorm(n, 0, me_sd)
  X_obs$x2 <- X_true$x2 + rnorm(n, 0, me_sd)
}

# -----------------------------------------------------------------------------
# 3. Construct the latent outcome signal
# -----------------------------------------------------------------------------

# Baseline linear signal shared across outcome families.
f_base <- 0.8 * x1 - 0.6 * x2 + 0.5 * x3 + 0.7 * x4 - 0.3 * x5

# Optional nonlinear component.
```

```r
# Introduces departures from a purely linear signal through quadratic,
# smooth oscillating, and threshold-type nonlinear terms.
f_nl <- switch(
  nonlinearity,
  "none" = rep(0, n),
  "moderate" = 0.5 * x1^2 - 0.4 * sin(x2),
  "strong" = 0.9 * x1^2 - 0.7 * sin(x2) + 0.5 * pmax(x3, 0)
)

# Optional interaction component.
# This is useful when studying omitted interactions or partial misspecification.
f_int <- switch(
  interactions,
  "none" = rep(0, n),
  "moderate" = 0.6 * x1 * x4 - 0.4 * x2 * x3,
  "strong" = 1.0 * x1 * x4 - 0.7 * x2 * x3 + 0.5 * x1 * x5
)

# Untreated latent mean.
mu0 <- f_base + f_nl + f_int

# Heterogeneous treatment effect.
# Treatment effects vary across units as a function of covariates.
tau <- 1.0 + 0.5 * x1 - 0.4 * x4 + 0.3 * (x5 == 2)

# Treated latent mean.
mu1 <- mu0 + tau

# ---------------------------------------------------------------------------
# 4. Generate treatment assignment
# ---------------------------------------------------------------------------

# Confounding controls how strongly treatment depends on outcome-relevant
# covariates. This creates imbalance on variables that also affect outcomes.
conf_strength <- switch(
  confounding,
  "none" = 0,
  "moderate" = 0.8,
  "strong" = 1.5
)

# Overlap controls how extreme the treatment probabilities become.
# Larger values push propensity scores closer to 0 or 1, even when the
# confounding component is weak or absent.
overlap_strength <- switch(
  overlap,
  "good" = 0,
  "moderate" = 1.1,
  "poor" = 1.8
)

# Confounding component:
# depends on outcome-relevant covariates and therefore induces selection bias
```

```r
  # when treatment is not randomized.
  lp_conf <- conf_strength * (0.8 * x1 - 0.6 * x2 + 0.5 * x4 - 0.3 * x5)

  # Overlap component:
  # depends primarily on a variable excluded from the outcome model, so it can
  # worsen common support without directly creating outcome confounding.
  # Because x6 is mean-zero in expectation, this component perturbs treatment
  # probabilities without systematically shifting them toward treatment or control.
  lp_overlap <- overlap_strength * x6

  # Total treatment-assignment linear predictor.
  lp_treat <- lp_conf + lp_overlap

  # True propensity score and realized treatment assignment.
  ps <- inv_logit(lp_treat)
  D <- rbinom(n, size = 1, prob = ps)

  # --------------------------------------------------------------------------
  # 5. Generate potential outcomes by outcome family
  # --------------------------------------------------------------------------
  if (outcome_type == "continuous") {
    # Outcome-specific error scale.
    # Heteroskedasticity is only used for continuous outcomes.
    # An exponential form is used for the conditional error standard deviation
    # so that heteroskedasticity varies smoothly with x1 while remaining
    # strictly positive for all units.
    sigma_eps <- switch(
      heteroskedasticity,
      "none" = rep(1, n),
      "moderate" = exp(0.35 * x1),
      "strong" = exp(0.7 * x1)
    )

    # Gaussian potential outcomes around latent means.
    Y0 <- mu0 + rnorm(n, 0, sigma_eps)
    Y1 <- mu1 + rnorm(n, 0, sigma_eps)

  } else if (outcome_type == "binary") {
    # Convert latent means to probabilities using the logistic link.
    p0 <- inv_logit(mu0)
    p1 <- inv_logit(mu1)

    # Bernoulli potential outcomes.
    Y0 <- rbinom(n, size = 1, prob = p0)
    Y1 <- rbinom(n, size = 1, prob = p1)

  } else if (outcome_type == "count") {
    # Convert latent means to Poisson rates.
    # Dividing by 3 tempers the rate scale so counts remain in a useful range
    # across a variety of signal settings.
    lambda0 <- exp(mu0 / 3)
    lambda1 <- exp(mu1 / 3)
    Y0 <- rpois(n, lambda0)
```

```
  Y1 <- rpois(n, lambda1)

} else if (outcome_type == "multinomial") {
  # Three-category multinomial outcome.
  # Category 1 is the reference category with linear predictor 0.
  eta0_mat <- cbind(
    rep(0, n),
    0.6 * mu0,
    -0.4 * mu0 + 0.5 * x2 - 0.3 * x4
  )

  eta1_mat <- cbind(
    rep(0, n),
    0.6 * mu1,
    -0.4 * mu1 + 0.5 * x2 - 0.3 * x4
  )

  # Convert category-specific linear predictors into probabilities.
  p0_mat <- softmax_vec(eta0_mat)
  p1_mat <- softmax_vec(eta1_mat)

  # Draw one category per row according to its multinomial probability vector.
  draw_multinom <- function(prob_mat) {
    apply(prob_mat, 1, function(p) sample(1:3, size = 1, prob = p))
  }

  Y0 <- draw_multinom(p0_mat)
  Y1 <- draw_multinom(p1_mat)
}

# Realized observed outcome under the switching equation.
Y <- ifelse(D == 1, Y1, Y0)

# ---------------------------------------------------------------------------
# 6. Introduce optional missingness
# ---------------------------------------------------------------------------

# Initialize missingness indicators.
miss_y <- rep(FALSE, n)
miss_x2 <- rep(FALSE, n)

if (missingness != "none") {
  # Missingness strength controls how strongly covariates/treatment predict
  # whether Y and x2 are observed.
  miss_strength <- switch(
    missingness,
    "mar_moderate" = 0.6,
    "mar_strong" = 1.1
  )

  # Outcome missingness depends on x1 and treatment.
  # Covariate missingness for x2 depends on x4 and x5.
  p_miss_y <- inv_logit(-2 + miss_strength * (0.8 * x1 + 0.6 * D))
```

```r
  p_miss_x2 <- inv_logit(-2.2 + miss_strength * (0.7 * x4 + 0.5 * x5))

  # Realize missingness indicators.
  miss_y <- rbinom(n, 1, p_miss_y) == 1
  miss_x2 <- rbinom(n, 1, p_miss_x2) == 1
}

# Build the observed dataset seen by the analyst.
dat <- X_obs
dat$D <- D
dat$ps_true <- ps
dat$Y <- Y

# Apply missingness after the observed data object is assembled.
dat$Y[miss_y] <- NA
dat$x2[miss_x2] <- NA

# ---------------------------------------------------------------------------
# 7. Compute true causal estimands
# ---------------------------------------------------------------------------

# For multinomial outcomes, a scalar treatment effect defined as Y1 - Y0
# is not directly meaningful in the same way as for the other outcome types,
# so the current implementation returns NA for these summary estimands.
if (outcome_type == "multinomial") {
  true_ate <- NA_real_
  true_att <- NA_real_
  true_atc <- NA_real_
} else {
  true_ate <- mean(Y1 - Y0)
  true_att <- mean((Y1 - Y0)[D == 1])
  true_atc <- mean((Y1 - Y0)[D == 0])
}

# Main return object: observed data, settings, and truth.
out <- list(
  data = dat,
  metadata = list(
    n = n,
    outcome_type = outcome_type,
    correlation = correlation,
    nonlinearity = nonlinearity,
    interactions = interactions,
    confounding = confounding,
    overlap = overlap,
    heteroskedasticity = heteroskedasticity,
    missingness = missingness,
    measurement_error = measurement_error
  ),
  truth = list(
    ate = true_ate,
    att = true_att,
    atc = true_atc
```

```r
    )
  )


  # Optionally return the underlying full data, including true covariates and
  # both potential outcomes. This is useful for validation and simulation studies.
  if (return_potential_outcomes) {
    out$full_data <- transform(
      X_true,
      D = D,
      ps_true = ps,
      Y0 = Y0,
      Y1 = Y1,
      Y = ifelse(D == 1, Y1, Y0),
      ite = if (outcome_type == "multinomial") NA_real_ else Y1 - Y0
    )
  }

  return(out)
}
```

Appendix: Validation Checks

This appendix reports a small set of implementation checks for `generate_sim_data()`. The goal is to verify that the principal components of the data-generating mechanism behave as intended.

Basic validation tables

Table 1: Basic validation summary for one representative simulation setting.

quantity	value
Rows in data	50,000
Rows in full_data	50,000
cor(x1, x2)	0.3506
cor(x1, x3)	0.3510
cor(x2, x3)	0.3513
Var(x1_obs - x1_true)	0.1594
Var(x2_obs - x2_true)	0.1610
Treatment prevalence	0.4953
Mean propensity score	0.4949
ATE	0.8714
ATT	0.9372
ATC	0.8067
Missing rate: Y	0.1515
Missing rate: x2	0.1592

Table 2: True propensity-score summaries across overlap settings with confounding fixed at none.

overlap	min	q1	median	mean	q3	max	pct_extreme
good	0.5000	0.5000	0.5000	0.5000	0.5000	0.5000	0.0000
moderate	0.0141	0.3239	0.5009	0.5003	0.6774	0.9931	0.0462
poor	0.0010	0.2307	0.5014	0.5003	0.7710	0.9997	0.2215

Table 3: Treatment-covariate dependence across confounding settings with overlap fixed at good.

confounding	cor_D_x1	cor_D_x2	cor_D_x4	cor_D_x5	mean_ps
none	-0.0060	-0.0011	-0.0015	-0.0014	0.5000
moderate	0.2217	-0.1132	0.0886	-0.0776	0.4935
strong	0.3404	-0.1774	0.1385	-0.1241	0.4897

Table A.1 provides a representative internal consistency check for the generator. The empirical covariate correlations closely match the target moderate correlation structure, the observed measurement-error variances are close to their programmed values, and the returned causal truth quantities are internally coherent. Missingness rates for the outcome and covariate x_2 are both around their expected levels under the moderate MAR-like setting.

Table A.2 confirms that the overlap control behaves as intended when examined separately from confounding. With confounding fixed at none, the true propensity scores remain concentrated around 0.5 under good overlap, become more dispersed under moderate overlap, and extend much closer to 0 and 1 under poor overlap, indicating progressively weaker common support.

Table A.3 confirms that the confounding control behaves as intended when examined separately from overlap. With overlap fixed at good, increasing the confounding setting strengthens treatment dependence on outcome-relevant covariates such as x_1, x_2, x_4, and x_5, while leaving the average treatment probability near 0.5.

Diagnostic plots

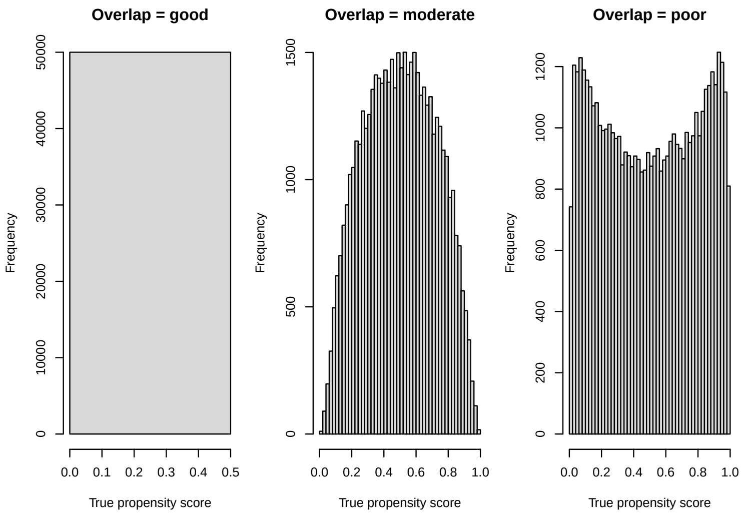

Figure A.1 confirms that the overlap control behaves as intended when separated from confounding. Under good overlap, the propensity scores remain concentrated near 0.5. Under moderate overlap, the distribution becomes more dispersed. Under poor overlap, the distribution places substantial mass nearer to 0 and 1, indicating pronounced deterioration in common support even in the absence of confounding.

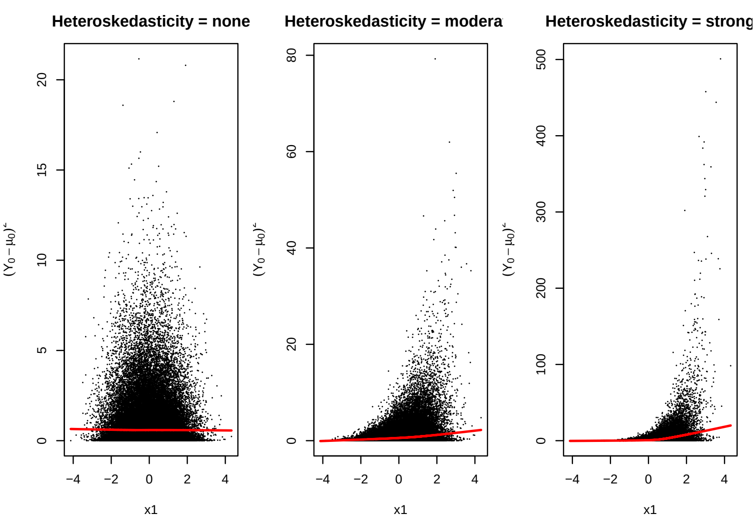

Figure A.2 confirms the programmed variance structure for continuous outcomes. Under homoskedasticity, the conditional spread of the residuals remains approximately constant across x_1. Under moderate heteroskedasticity, residual variability increases with x_1. Under strong heteroskedasticity, this increase becomes more pronounced, producing a clear widening fan shape.

Summary

Taken together, these checks indicate that the main components of the generator behave as intended: the continuous covariates have the desired correlation structure, the overlap control now produces increasingly extreme propensity scores independently of confounding, the confounding control increases treatment dependence on outcome-relevant covariates while overlap is held fixed, the heteroskedasticity control changes the variance pattern as designed, and the returned truth quantities are internally consistent with the simulated potential outcomes.